

Major Lab Report 8-9: Intro to FPGAs and VHDL Programming, Design of Code
Converter, Adders, and Traffic Light Controller Using FPGAs

Paul Mosincat, Anthony Zavala ECE-218-L03

TA: Bo Yang Chen

Lab Date: November 21, 2025

Due Date: November 30, 2025

I. Abstract

Paul Mosincat: Anthony and I split the work evenly throughout both of the labs and the different sub-experiments. I helped set up the FPGA code for each of the labs through the VHDL. The code was tailored to perform the specific task as outlined in the manual. Additionally, to write the code for each of the experiments I was tasked with debugging the code to ensure that the FPGA was able to perform the function dedicated to that specific section in the lab and the FPGA board worked as intended. Lastly, I documented the code for each of the different sections.

Anthony Zavala: Paul and I split the work evenly throughout both of the labs and the different sub-experiments. While Paul was setting up the VHDL code for each of the experiments, I was building the breadboard to perform the specific task in the lab. This includes setting up the FPGA on the breadboard, wired components like the seven-segment display and pushbuttons. Additionally, I ensured that each of the wired components was properly connected to the FPGA chip to ensure that the code worked and our breadboard would perform the given function whether it was adding or doing the traffic control.

II. Introduction

This final pair of labs was designed to introduce students to the use of FPGAs (Field Programmable Gate Arrays) which were reconfigurable devices that allowed the user to implement digital logic through different programmable codes uploaded to the chip. These 2 labs focused on logic concepts that we have been using throughout the semester but rather than using TTL components to perform a specific task we were able to use the VHDL code and FPGA to perform these tasks in a more efficient manner.

The main objectives of this lab focused on becoming more familiar with FPGA programming and design process as well as the using the CMOD A7-35T Artix-7 board and Xilinx's Vivado Design Suite to achieve a working circuit. Through these 2 labs we completed a wide range of guided experiments such as a ripple-carry adder and a traffic light controller to understand the design and implementation process for the FPGA are more efficient than the traditional combinational circuits we used throughout the semester.

Although these experiments were rather simple compared to industry standards, these experiments proved to be rather challenging in the design and

implementation process. At the end of these 3-week long experiments, we were able to gain basic and hands-on understanding of coding, debugging, and testing, which professionals use when designing more complicated circuits.

III. Experiments, Results, and Interpretations

Lab 8 Experiment 1: Basic Logic Design with FPGAs and VHDL Programming Objective

Before beginning the experiments in the Lab 9 sections, we had to understand the process of what went into developing the FPGA and the VHDL code. We had to set up the breadboard to ensure that the overall functionality from Lab 8 was transferred to Lab 9 when performing the experiments. Each group received a CMOD A7-35T FPGA board and when the circuit was properly wired to outline the diagram 8.8 from the lab manual we were able to start understanding the coding aspect of these labs. Once the physical setup was complete, we launched the Vivado Design Suite and created new projects using the instructions and code provided in the lab manual which served as the basic building block for the following experiments. Overall the initial Lab 8 started as the basic understanding of the FPGA board and the VHDL code. All of the code and circuits will be placed at the end of the lab report in the Appendix for easier visibility and clarity.

Lab 9 Experiment 1: Code Conversion: Excess 3 and Hexadecimal Encoder of 4-Bit Binary Numbers

This experiment focused on introducing a more complex converter by using a simple encoder and 2-way converter. The goal of the first part of the experiment was simply to use VHDL to simulate the conversion from Natural BCD to Excess-3. The second part of this experiment was to use the already existing conversion and create a 2-way conversion when the signal is low it implements the already existing converter of Natural to Excess-3 and when the signal is high FPGA converts Excess-3 to Natural. Unlike the previous labs we completed in this

course and are going to complete in the following experiments, this experiment solely depended on the VHDL code to function properly.

Materials:

- Vivado Design Suite
- VHDL (files provided in lab manual)
- Breadboard from Lab 8 with FPGA chip

Procedure:

1. Using the truth tables and the code that is already provided, we built the original converter of Natural to Excess-3 by writing VHDL code.
2. Using the current code we created additional VHDL code that was compatible with converting both ways from Natural to Excess-3 and vice versa.
3. We then synthesized, implemented, generated bit-stream, and programmed the FPGA chip with the functional code.
4. We then tested the VHDL coder conversion with the seven segment display and buttons.

Results:

The simulations confirmed that the basic converter and the universal code converter both worked correctly with only a minor error being that the seven-segment display would not display the proper conversion value when the specific buttons were pressed. This error persisted after working with the TA to change the wiring and changing display. This could indicate a problem with the breadboard itself or there was a minor problem with the code. Overall the VHDL code proved that the simple and universal code convert worked as intended and was able to perform the conversion from Natural to Excess-3 depending on the signal even if there was a minor error present.

Interpretation:

This experiment highlighted how much more effective the FPGA programming and chip are when we assigned a specific task such as performing a conversion. Compared to the similar lab we previously did using logic gates to perform the conversion the FPGA showed that it was much more versatile in performing the conversion. Additionally, the overall code was easier to troubleshoot when there was an error as opposed to the logic gates one where you had to find the specific component that was malfunctioning. Overall, this experiment showed the benefits of having a much more effective means of performing a task and showed how a dynamic circuit is able to function solely from VHDL code.

Lab 9 Experiment 2: Four-Bit Ripple-Carry Adder/Subtractor

This experiment involved creating a four-bit ripple-carry adder/subtractor using VHDL and testing the functionality through the seven-segment display and hardware implementation. The objective of this experiment was to understand how the FPGA is able to perform both addition and subtraction using an input signal as well as being built with a full-adder. By incorporating VHDL code that serves as the input signal we are able to perform both addition and subtraction.

Materials:

- Vivado Design Suite
- VHDL (files provided in lab manual)
- Breadboard with FPGA chip(similar to Lab 8 Figure 8.8)
- 4 additional buttons

Procedure:

1. Using the provided VHDL files we created a new project in the Vivado Design Suite and created code that performs addition and subtraction based on the given input from the buttons.

2. We then synthesized, implemented, generated bit-stream, and programmed the FPGA chip with the functional code.
3. Buttons were used to determine the binary input for the FPGA chip to perform addition or subtraction and the following results were then displayed on the seven-segment display.
4. We debugged the code if there were any errors or issues with the FPGA performing addition or subtraction.

Results:

The simulations confirmed that the FPGA was able to perform the addition and subtraction which was then displayed on the seven-segment display. There was a minor error similar to the previous experiment in Lab 9. The code was not working properly as intended which caused the display to improperly display the result when a specific binary input was pressed. Overall, the code had the necessary logic to perform the given addition or subtraction but there were problems between the display and the code which caused error when having a specific binary input.

Interpretation:

Based on the previous experiment that was completed in this semester similar to this ripple-adder for addition and subtraction, the FPGA chip and the VHDL code showed more versatility and effectiveness in performing arithmetic operations. In the previous experiments in this course creating a simple ripple-adder required logic gates and specific wiring toward a switch to get the circuit to perform an operation. For subtraction we needed to use the XOR gate to ensure that the circuit could perform subtraction. This experiment showed that the FPGA chip and the VHDL code is more efficient in performing arithmetic calculations and reducing the use of logic gates. Overall, this experiment added onto the experience of getting more hands on experience in digital systems but also reinforced the idea that VHDL and FPGA can perform arithmetic operation more accurately and on a larger scale showing that it is more efficient than traditional gates.

Lab 9 Experiment 3: Traffic Light Controller

This experiment focused on building a simplified real world traffic light system. The objective was to show finite state machines can be applied to systems such as traffic lights. This experiment focused on using a traffic light controller that switches from one light to next based upon the current state and clock timer. In a sample case, the traffic light would start on green and then after 5 seconds have elapsed the traffic light would then switch to yellow and it would repeat this pattern until red was reached. This system was managed through the FPGA chip's clock and programmed with the VHDL code to incorporate this clock which caused the change in states(lights). The traffic light system for both parts of the experiment was incorporated with VHDL programmable code and based off of the given flowcharts from the lab manual.

Materials:

- Vivado Design Suite
- VHDL (files provided in lab manual)
- Breadboard with FPGA chip(similar to Lab 8 Figure 8.8)
- 2 LEDS of red, yellow, and green(6 total)

Procedure:

1. We began by creating a new project and importing the VHDL files that are provided from the lab manual in this specific part of the experiments.
2. We then created code based upon the flowcharts for each of the 2 parts in the traffic control experiment.
3. We then synthesized, implemented, generated bit-stream, and programmed the FPGA chip with the functional code.
4. We then observed the changing of the LED's which represents the transition from 1 state to the next. We did this specific to each parts and corresponding flow chart.

Results:

The simulations confirmed that the FPGA was able to perform the specific transitions from one state to the next given the internal clock input. This worked for both of the parts in the experiment based on the given flowcharts. This particular experiment had relatively no errors due to circuits and LED's working as intended. The only minor error was the code and having the to fix the clock pulse. Overall, the circuit was able to simulate an actual traffic control system.

Interpretation:

Based on the previous finite state machine that we were tasked with building in the previous experiments this semester required a large number of logic gates and flip-flops to perform the transition between each state. Additionally, this required a significant amount of wires connecting each of the gates and flip-flops. As mentioned before, the FPGA and the VHDL code again show the effectiveness and efficiency when performing the specific task such as the modified traffic control system in this part of the experiment. Additionally, it shows the simplicity of a circuit compared to the previous circuit we built. Lastly, this experiment shows the effectiveness of the FPGA and VHDL in control logic systems that are dependent on timing and require accuracy such as modern traffic control systems.

IV. Theoretical Questions

1. What is debouncing in hardware and why is it important?

Debouncing in hardware refers to the process of eliminating unwanted signals which cause glitches or noise within a given digital system. An example of this would be the buttons used in Lab 9 Experiment 2 with the adders where multiple buttons can be pushed to create noise and cause the system confusion in what specific task to perform. Debouncing is important for hardware and any

digital system as it allows for one constant signal and removes any error with multiple inputs being detected.

2. What are FPGAs mainly used for and how do they differ from ASICs?

The main difference between the FPGA's and ASIC's is the functionality and the types of projects each piece is used for. First the functionality, the FPGA is a programmable and reconfigurable chip that uses logic implementation to give functionality to the circuit whereas the ASIC is custom made for a single function in mind. Second, the FPGA can be used for minor projects as the ones performed in these experiments where the ASIC can be used for larger scale projects.

3. Why might you choose an FPGA for your design?

You might use an FPGA for your design or any future projects when there is something that might need to be changed in the future or the project requires flexibility. An FPGA is good as it has the ability to be reprogrammable which gives it a versatile incorporation into a project that requires constant changing or updates. Overall you might use the FPGA for a design that needs to be tailored to specific tasks and changed in the future similar to some of the experiments performed in this lab.

4. What are some of the biggest industry applications of FPGAs? Academic applications?

FPGAs are widely used in industries such as telecommunications, automotive, and a wide range of other industries. These industries require the FPGA as it is able to perform data processing and is easily reprogrammable which also allows companies to design FPGA for their specific goal or task they have in mind. In terms of academic applications, a majority of schools including IIT use the FPGA as a way for students to get hands-on experience for digital systems and design circuits around it.

5. What are the differences between VHDL and Verilog? Are there any other programming languages that can be used for hardware/firmware design?

As both VHDL and Verilog are used for hardware programmable language there lies one major difference. VHDL is more structured and used for more intense/larger projects whereas Verilog is tailored for smaller projects as it is easier to write with less structure. Yes there are more modern versions such as MyHDL which is a Python programmable language for firmware design. These modern features offer similar features. Overall, the type of program used in hardware design depends on the type of projects as well as other factors.

6. Why do some FPGAs cost so much?

There are a wide variety of factors which cause some of the FPGAs to cost more than others. The first factor being the components that each FPGA is built with. For example if a FPGA has a larger processor it's going to cost more than one that has a smaller processor. The specs of each FPGA determine the cost of it. Another factor is the amount of each FPGA produced affects the cost as one that is limited will cost more than an FPGA that's abundant.

V. Conclusion

The series of experiments conducted in this lab highlighted how FPGAs are able to perform a function/task more effectively than traditional logic gates and flip-flops. Through the use of Vivado and VHDL we were able to create programmable hardware that was able to perform conversions, arithmetic operations, and based traffic control systems. Each part from developing the code to generating the bitstream played a part in creating a fully functional digital

system. Overall, we were able to gain hands-on experience and learned how to implement a proper digital system with VHDL and FPGA chips.

Lab 8 Experiment 1: Basic Logic Design with FPGAs and VHDL Programming Objective

Lab 8 served as the introduction into the digital system and hardware design. In the specific section we learned the basics of VHDL programming and how to set the Vivado. We learned the basic setup for the FPGA chip and how to map the output of the chip to specific components to perform a given task/function. This part of this experiment gave us hands-on experience in learning the basics before we began performing more in-depth experiments.

Lab 9 Experiment 1: Code Conversion: Excess 3 and Hexadecimal Encoder of 4-Bit Binary Numbers

The initial experiment for Lab 9 focused on integrating the components and initial circuit from Lab 8. We used push buttons and a seven-segment display to ensure functionality before incorporating the conversion factor. This part showed that the FPGA chip and VHDL code reduced the overall components of the circuit. The next 2 parts for this experiment focused on the conversion between Natural BCD to Excess-3 BCD and a universal converter between both. We developed and programmed the FPGA for both the simple conversion and universal conversion. We did end up having a minor error but the overall logic of the code showed that the conversion did happen. Overall, this experiment highlights how efficient and effective the FPGA and VHDL are at performing a given task and showed us the beginning of changeable digital systems.

Lab 9 Experiment 2: Four-Bit Ripple-Carry Adder/Subtractor

This experiment focused on creating a new project based around ripple-carry for addition and subtractions. We used buttons that represent the 4-bits for 2 numbers and then displayed the result on the seven-segment display. The VHDL code performs both operations. The FPGA showed the capabilities of performing arithmetic operations. Additionally, compared to previous ripple adder, the FPGA chip again reduced the amount of components required to perform arithmetic operations. Overall, this experiment shows the benefits of FPGA performing arithmetic operation and reinforcing the idea of efficiency/effectiveness in digital design.

Lab 9 Experiment 3: Traffic Light Controller

This experiment focused on implementing a modified traffic control system. We used the same process of creating a new project in Vivado and developing code based on the flowcharts provided to create a traffic control system that switches between the different states(lights). This experiment showed the real world application of the FPGA and the development process for any real world digital system. Additionally, it showed the impact the FPGA and VHDL has on developing hardware. Overall, this experiment highlights the importance of the FPGA and VHDL in the real world and how professionals develop more intricate digital systems.

VI. Appendix

Code,Circuits, and Diagrams

Lab 8 Experiment 1: Basic Logic Design with FPGAs and VHDL Programming

Objective

Code:

```

--alog inputs
[get_ports { pio1 }],
[get_ports { pio2 }],
[get_ports { button[0]
[get_ports { button[1]
[get_ports { button[2]
[get_ports { button[3]
[get_ports { pio7 }],
[get_ports { pio8 }],
[get_ports { pio9 }],
[get_ports { pio10 }],

22 library IEEE;
23 use IEEE.STD_LOGIC_1164.ALL;
24
25 -- Uncomment the following library declaration if using
26 -- arithmetic functions with Signed or Unsigned values
27 --use IEEE.NUMERIC_STD.ALL;
28
29 -- Uncomment the following library declaration if instantiating
30 -- any Xilinx leaf cells in this code.
31 --library UNISIM;
32 --use UNISIM.VComponents.all;
33
34 entity Lab_8_File is
35     Port ( clk : in STD_LOGIC;
36           button: in STD_LOGIC_VECTOR (3 downto 0);
37           display: out STD_LOGIC_VECTOR (8 downto 0);
38           led : out STD_LOGIC
39     );
40 end Lab_8_File;
41
42 architecture Behavioral of Lab_8_File is
43     begin
44         process (clk)
45             variable cnt : integer range 0 to 35000000;
46             begin
47                 if (clk' event and clk = '1') then
48                     if (cnt = 35000000) then
49                         cnt := 0;
50                     elsif (cnt > 25000000 and cnt < 35000000) then
51                         led <= '1';
52                         cnt := cnt + 1;
53                     else
54                         led <= '0';
55                         cnt := cnt + 1;
56                     end if;
57                     -- Blink the LED at ~0.5 Hz
58                     if cnt < 17500000 then
59                         led <= '1';
60                     else
61                         led <= '0';
62                     end if;
63
64                     -- Display mirrors button presses (for testing)
65                     display(3 downto 0) <= button; -- shows which buttons are pressed
66                     display(8 downto 4) <= "0000"; -- remaining LEDs off
67                 end if;
68             end process;
69 end Behavioral;
70

```

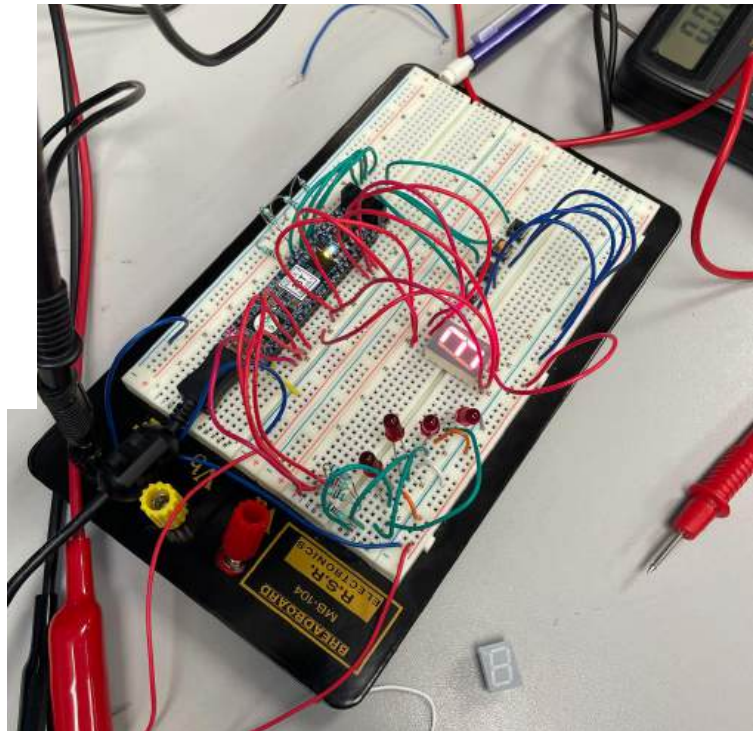
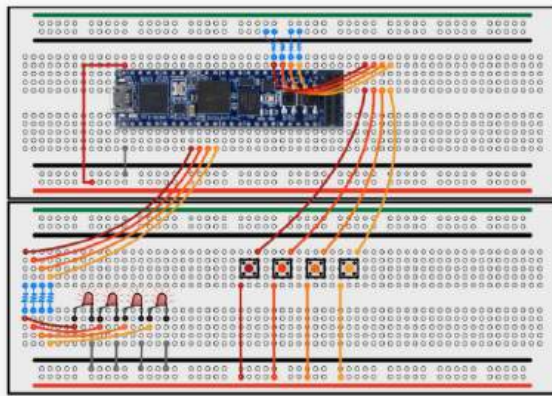
```

33 } [get_ports { pio26 }]; #
3 } [get_ports { led }]; #IO
3 } [get_ports { led }]; #IO
3 } [get_ports { led }]; #IO
3 } [get_ports { led }]; #IO
33 } [get_ports { pio31 }]; #
33 } [get_ports { pio32 }]; #
33 } [get_ports { pio33 }]; #
3 } [get_ports { display[0]
3 } [get_ports { display[1]
3 } [get_ports { display[2]
3 } [get_ports { display[3] }
33 } [get_ports { pio38 }]; #
33 } [get_ports { pio39 }]; #
3 } [get_ports { display[4] }
33 } [get_ports { pio41 }]; #
33 } [get_ports { pio42 }]; #
3 } [get_ports { display[5] }
3 } [get_ports { display[6] }
3 } [get_ports { display[7]
3 } [get_ports { display[8] }
33 } [get_ports { pio47 }]; #
33 } [get_ports { pio48 }]; #

```

Diagram/Circuit:

Figure 8.7: Schematic Setup for FPGA Labs



Lab 9 Experiment 1: Code Conversion: Excess 3 and Hexadecimal Encoder of 4-Bit Binary Numbers

Code:
library IEEE;

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity top_level is
```

```
  Port (
```

```
    clk    : in  STD_LOGIC;
```

```
    buttons : in  STD_LOGIC_VECTOR(3 downto 0);
```

```
    seg     : out STD_LOGIC_VECTOR(6 downto 0)
```

```
  );
```

```
end top_level;
```

```
architecture Behavioral of top_level is
```

```
  signal clean_btns : STD_LOGIC_VECTOR(3 downto 0);
```

```
  signal Y : STD_LOGIC_VECTOR(3 downto 0);
```

```
begin
```

```
  debouncer_inst : entity work.dbntog
```

```
    port map (
```

```
      clk    => clk,
```

```
      input  => buttons,
```

```
      output => clean_btns
```

```
    );
```

```
  converter_inst : entity work.simpleCodeConv
```

```
    port map (
```

```
      A => clean_btns,
```

```
      Y => Y
```

```
    );
```

```
  display_inst : entity work.segmentDecoder
```

```
    port map (
```

```
      data => Y,
```

```
      seg  => seg
```

```
    );
```

```
end Behavioral;
```

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity simpleCodeConv is
```

```
  Port (
```

```
    A : in  STD_LOGIC_VECTOR (3 downto 0);
```

```

        Y : out STD_LOGIC_VECTOR (3 downto 0)
    );
end simpleCodeConv;

architecture Behavioral of simpleCodeConv is
begin
    Y <= std_logic_vector(unsigned(A) + 3);
    Y <= std_logic_vector(unsigned(A) - 3);
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity segmentDecoder is
    Port (
        data : in  STD_LOGIC_VECTOR(3 downto 0);
        seg  : out STD_LOGIC_VECTOR(6 downto 0)
    );
end segmentDecoder;

architecture Behavioral of segmentDecoder is
begin
    process(data)
    begin
        case data is
            when "0000" => seg <= "1000000"; -- 0
            when "0001" => seg <= "1111001"; -- 1
            when "0010" => seg <= "0100100"; -- 2
            when "0011" => seg <= "0110000"; -- 3
            when "0100" => seg <= "0011001"; -- 4
            when "0101" => seg <= "0010010"; -- 5
            when "0110" => seg <= "0000010"; -- 6
            when "0111" => seg <= "1111000"; -- 7
            when "1000" => seg <= "0000000"; -- 8
            when "1001" => seg <= "0010000"; -- 9

            -- Added values for A-F
            when "1010" => seg <= "0001000"; -- A
            when "1011" => seg <= "0000011"; -- b
            when "1100" => seg <= "1000110"; -- C
            when "1101" => seg <= "0100001"; -- d
            when "1110" => seg <= "0000110"; -- E
            when "1111" => seg <= "0001110"; -- F
        end case;
    end process;
end Behavioral;

```



```

        when others => seg <= "1111111"; -- blank
    end case;
end process;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity dbntog is
    Port (
        clk    : in  STD_LOGIC;
        input   : in  STD_LOGIC_VECTOR (3 downto 0);
        output  : out STD_LOGIC_VECTOR (3 downto 0)
    );
end dbntog;

architecture Behavioral of dbntog is
begin
    process(clk)
    begin
        if rising_edge(clk) then
            output <= input; -- simple debouncer version from lab
        end if;
    end process;
end Behavioral;

```

Diagrams/Truth Table/Circuit:

Table 9.1: BCD Codes

Decimal	Natural BCD				Excess-3			
	A ₃	A ₂	A ₁	A ₀	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

Figure 9.1: Converter Circuit Block Diagrams

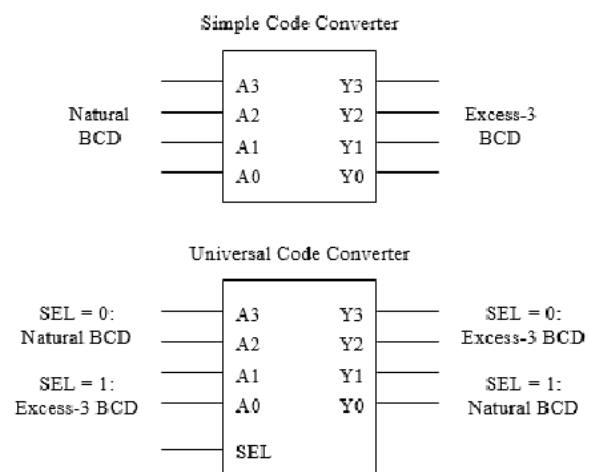
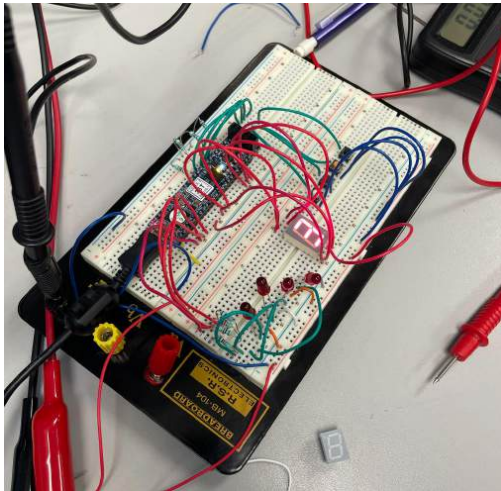


Figure 1 – Converter Circuit Block Diagrams.

Lab 9 Experiment 2: Four-Bit Ripple-Carry Adder/Subtractor

Code:

```
library ieee;  
use ieee.std_logic_1164.all;
```



```
entity full_adder is  
  port (  
    a  : in std_logic;  
    b  : in std_logic;  
    c_in : in std_logic;  
    s   : out std_logic;  
    c_out : out std_logic  
  );  
end full_adder;
```

architecture behav of full_adder is

```
  signal axb : std_logic;  
  signal c1  : std_logic;  
  signal c2  : std_logic;  
  signal c3  : std_logic;
```

```
begin  
  axb <= a xor b;  
  s   <= axb xor c_in;
```

```
c1 <= a and b;  
c2 <= a and c_in;  
c3 <= b and c_in;
```

```
c_out <= c1 or c2 or c3;
```

```
end behav;
```

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity adder_subtractor is  
  port (  
    A : in std_logic_vector(3 downto 0);  
    B : in std_logic_vector(3 downto 0);  
    mode : in std_logic;      -- 0 = add, 1 = subtract  
    S : out std_logic_vector(3 downto 0);  
    c_out : out std_logic  
  );  
end adder_subtractor;
```

```
architecture rtl of adder_subtractor is
```

```
  component ripple_carry_adder is  
    generic (WIDTH : natural := 4);  
    port (  
      A : in std_logic_vector(WIDTH-1 downto 0);  
      B : in std_logic_vector(WIDTH-1 downto 0);  
      c_in : in std_logic;  
      S : out std_logic_vector(WIDTH-1 downto 0);  
      c_out : out std_logic  
    );  
  end component;
```

```
  signal B_mod : std_logic_vector(3 downto 0);
```

```
begin
```

```
  B_mod <= B xor (mode & mode & mode & mode);
```

```
  ADDER : ripple_carry_adder  
    port map(  
      A => A,  
      B => B_mod,  
      c_in => c_in,  
      S => S,  
      c_out => c_out
```

```

    A    => A,
    B    => B_mod,
    c_in => mode,
    S    => S,
    c_out => c_out
);

```

```
end rtl;
```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```
entity top_level is
```

```

    Port (
        clk    : in  STD_LOGIC;
        btns   : in  STD_LOGIC_VECTOR(7 downto 0); -- 8 buttons total
        seg    : out STD_LOGIC_VECTOR(6 downto 0) -- 7-segment
    );

```

```
end top_level;
```

```
architecture Behavioral of top_level is
```

```
-- COMPONENTS USED
```

```

component dbntog
    port(
        clk : in  std_logic;
        btn : in  std_logic;
        dbnc : out std_logic
    );

```

```
end component;
```

```
component adder_subtractor
```

```

    port(
        A    : in  std_logic_vector(3 downto 0);
        B    : in  std_logic_vector(3 downto 0);
        mode : in  std_logic;
        S    : out std_logic_vector(3 downto 0);
        c_out : out std_logic
    );

```

```
end component;
```

```
component segmentDecoder1
```

```

    port(
        data : in  std_logic_vector(3 downto 0);

```

```
        seg : out std_logic_vector(6 downto 0)
    );
end component;
```

```
-- INTERNAL SIGNALS
```

```
signal A_clean : std_logic_vector(3 downto 0);
signal B_clean : std_logic_vector(3 downto 0);
signal mode_clean : std_logic;
```

```
signal Sum : std_logic_vector(3 downto 0);
signal Carry : std_logic;
```

```
begin
```

```
-----
-- DEBOUNCE ALL BUTTONS
-----
```

```
-- A inputs use btns(3 downto 0)
```

```
GEN_A : for i in 0 to 3 generate
```

```
    Adeb : dbntog
```

```
    port map(
```

```
        clk => clk,
```

```
        btn => btns(i),
```

```
        dbnc => A_clean(i)
```

```
    );
```

```
end generate;
```

```
-- B inputs use btns(7 downto 4)
```

```
GEN_B : for i in 0 to 3 generate
```

```
    Bdeb : dbntog
```

```
    port map(
```

```
        clk => clk,
```

```
        btn => btns(i+4),
```

```
        dbnc => B_clean(i)
```

```
    );
```

```
end generate;
```

```
-- mode select uses btns(0) or whichever button you want
```

```
modedeb : dbntog
```

```
port map(
```

```
    clk => clk,
```

```
    btn => btns(0),
```

```
    dbnc => mode_clean
```

```
);
```

-- ADDER / SUBTRACTOR

```
ADDER_INST : adder_subtractor
  port map(
    A    => A_clean,
    B    => B_clean,
    mode => mode_clean,
    S    => Sum,
    c_out => Carry
  );
```

-- DISPLAY SUM ON 7-SEGMENT

```
DISP : segmentDecoder1
  port map(
    data => Sum,
    seg  => seg
  );
```

end Behavioral;

Error for synthesis

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity dbntog is

Port (

clk : in STD_LOGIC;

btn : in STD_LOGIC;

dbnc : out STD_LOGIC

);

end dbntog;

architecture Behavioral of dbntog is

constant debounce_limit : integer := 1000000;

signal count : integer := 0;

signal btn_state : std_logic := '0';

signal btn_sync : std_logic := '0';

```

begin

    process(clk)
    begin
        if rising_edge(clk) then

            -- synchronize button to avoid metastability
            btn_sync <= btn;

            if btn_sync /= btn_state then
                count <= 0;
                btn_state <= btn_sync;
            elsif count < debounce_limit then
                count <= count + 1;
            end if;

            if count = debounce_limit then
                dbnc <= btn_state;
            end if;

        end if;
    end process;

end Behavioral;

Correct Top File
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity top_level is
    Port (
        clk : in  STD_LOGIC;
        btns : in  STD_LOGIC_VECTOR(7 downto 0); -- 8 push buttons
        seg : out STD_LOGIC_VECTOR(6 downto 0) -- 7-seg display
    );
end top_level;

architecture Behavioral of top_level is

    -----
    -- Component declarations
    -----

    component dbntog is

```

```

Port (
    clk : in STD_LOGIC;
    btn : in STD_LOGIC;
    dbnc : out STD_LOGIC
);
end component;

```

```

component adder_subtractor is
port (
    A : in std_logic_vector(3 downto 0);
    B : in std_logic_vector(3 downto 0);
    mode : in std_logic;           -- 0 = add
    S : out std_logic_vector(3 downto 0);
    c_out : out std_logic
);
end component;

```

```

component segmentDecoder1 is
port (
    data : in std_logic_vector(3 downto 0);
    seg : out std_logic_vector(6 downto 0)
);
end component;

```

```

-----
-- Internal signals
-----

```

```

signal A_clean : std_logic_vector(3 downto 0); -- debounced A buttons
signal B_clean : std_logic_vector(3 downto 0); -- debounced B buttons
signal Sum : std_logic_vector(3 downto 0); -- adder output
signal Carry : std_logic; -- not displayed
constant mode : std_logic := '0'; -- always ADD

```

```

begin

```

```

-----
-- Debounce A inputs (btns(3 downto 0))
-----

```

```

GEN_A : for i in 0 to 3 generate
    Adeb : dbntog
    port map (
        clk => clk,
        btn => btns(i),
        dbnc => A_clean(i)
    )
end generate

```



```
);  
end generate;
```

```
-----  
-- Debounce B inputs (btns(7 downto 4))  
-----
```

```
GEN_B : for i in 0 to 3 generate  
  Bdeb : dbntog  
    port map (  
      clk => clk,  
      btn => btns(i + 4),  
      dbnc => B_clean(i)  
    );  
end generate;
```

```
-----  
-- 4-bit ripple-carry adder (adder_subtractor with mode = '0')  
-----
```

```
ADDER_INST : adder_subtractor  
  port map (  
    A  => A_clean,  
    B  => B_clean,  
    mode => mode, -- '0' => addition only  
    S  => Sum,  
    c_out => Carry  
  );
```

```
-----  
-- Show Sum (0–15) on 7-segment display  
-----
```

```
DISP : segmentDecoder1  
  port map (  
    data => Sum,  
    seg => seg  
  );
```

```
end Behavioral;
```

Ripple Adder:

```
library ieee;  
use ieee.std_logic_1164.all;
```

```

entity ripple_carry_adder is
  generic (WIDTH : natural := 4);
  port (
    A   : in  std_logic_vector(WIDTH-1 downto 0);
    B   : in  std_logic_vector(WIDTH-1 downto 0);
    c_in : in  std_logic;
    S   : out std_logic_vector(WIDTH-1 downto 0);
    c_out : out std_logic
  );
end ripple_carry_adder;

```

architecture rtl of ripple_carry_adder is

```

  component full_adder is
    port (
      a   : in  std_logic;
      b   : in  std_logic;
      c_in : in  std_logic;
      s   : out std_logic;
      c_out : out std_logic
    );
  end component;

```

```

  signal carry : std_logic_vector(WIDTH downto 0);

```

```

begin

```

```

  -----
  -- Initial carry-in
  -----

```

```

  carry(0) <= c_in;

```

```

  -----
  -- Generate WIDTH full-adders in a chain (ripple)
  -----

```

```

  GEN_FA : for i in 0 to WIDTH-1 generate

```

```

    FA_i : full_adder

```

```

      port map(

```

```

        a   => A(i),

```

```

        b   => B(i),

```

```

        c_in => carry(i),

```

```

        s   => S(i),

```

```

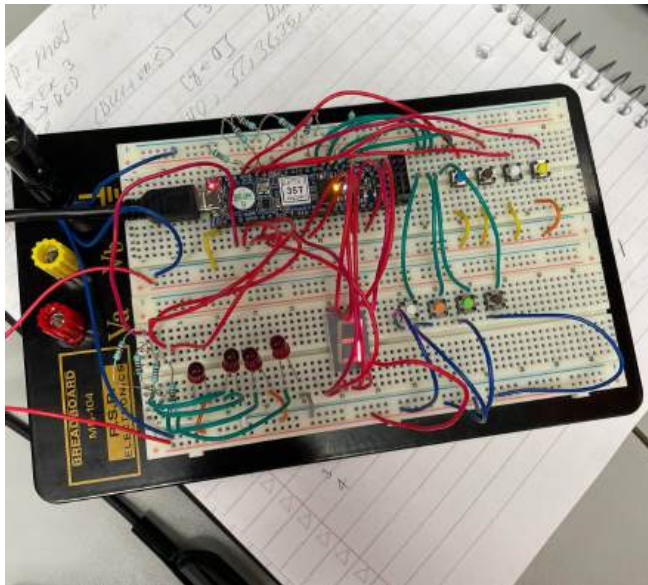
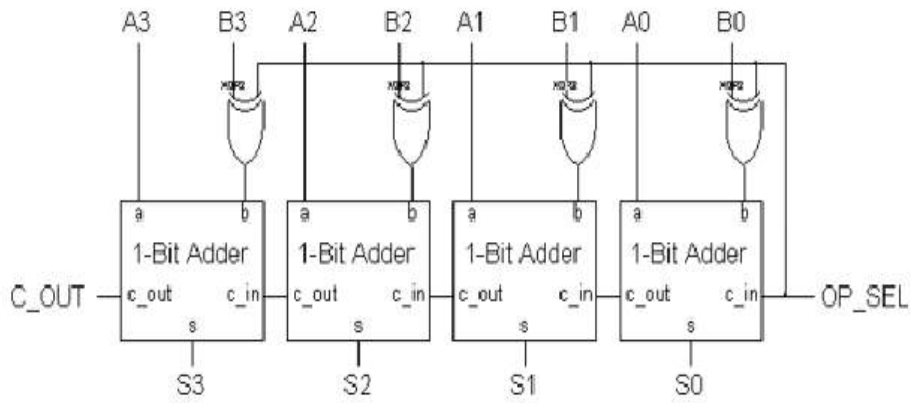
        c_out => carry(i+1)
      );
    end GEN_FA;
end rtl;

```

```
    );  
end generate;  
  
-----  
-- Final carry-out  
-----  
c_out <= carry(WIDTH);  
  
end rtl;
```

Schematics/Diagrams/Circuit:

Figure 9.2: Schematic Diagram for Four-Bit Ripple-Carry Adder/Subtractor



Lab 9 Experiment 3: Traffic Light Controller

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
```

```
entity trafficControlFSM is
port (
    clk : in std_logic;
```

```

        ns_r, ns_y, ns_g : out std_logic;
        ew_r, ew_y, ew_g : out std_logic
    );
end trafficControlFSM;

```

architecture behav of trafficControlFSM is

```

-----
-- Traffic Light FSM States
-----

```

```

type state_type is (s0, s1, s2, s3, s4, s5);
signal state, next_state : state_type;

```

```

-----
-- Timer (counts seconds)
-----

```

```

signal timer : integer := 0;
signal timer_reset : std_logic := '0';

```

```

-----
-- Clock divider : create 1 Hz enable pulse from 100 MHz clock
-----

```

```

signal clk_en : std_logic := '0';
signal div_counter : integer := 0;
constant DIV_MAX : integer := 100_000_000 - 1; -- 100 MHz → 1 Hz

```

begin

```

-----
-- Clock Divider (100 MHz → 1 pulse per second)
-----

```

```

clk_divider : process(clk)
begin
    if rising_edge(clk) then
        if div_counter = DIV_MAX then
            div_counter <= 0;
            clk_en <= '1';
        else
            div_counter <= div_counter + 1;
            clk_en <= '0';
        end if;
    end if;
end process;

```

```
-- State register + timer (only when clk_en = 1)
```

```
sync_process : process(clk)
begin
    if rising_edge(clk) then
        if clk_en = '1' then
            state <= next_state;

            if timer_reset = '1' then
                timer <= 0;
            else
                timer <= timer + 1;
            end if;
        end if;
    end if;
end process;
```

```
-- Output decode
```

```
output_decode : process(state)
begin
    case state is

        -- s0: NS green, EW red
        when s0 =>
            ns_r <= '0'; ns_y <= '0'; ns_g <= '1';
            ew_r <= '1'; ew_y <= '0'; ew_g <= '0';

        -- s1: NS yellow, EW red
        when s1 =>
            ns_r <= '0'; ns_y <= '1'; ns_g <= '0';
            ew_r <= '1'; ew_y <= '0'; ew_g <= '0';

        -- s2: all red
        when s2 =>
            ns_r <= '1'; ns_y <= '0'; ns_g <= '0';
            ew_r <= '1'; ew_y <= '0'; ew_g <= '0';

        -- s3: NS red, EW green
        when s3 =>
```

```

    ns_r <= '1'; ns_y <= '0'; ns_g <= '0';
    ew_r <= '0'; ew_y <= '0'; ew_g <= '1';

-- s4: NS red, EW yellow
when s4 =>
    ns_r <= '1'; ns_y <= '0'; ns_g <= '0';
    ew_r <= '0'; ew_y <= '1'; ew_g <= '0';

-- s5: all red
when s5 =>
    ns_r <= '1'; ns_y <= '0'; ns_g <= '0';
    ew_r <= '1'; ew_y <= '0'; ew_g <= '0';

-- fallback
when others =>
    ns_r <= '1'; ns_y <= 'X'; ns_g <= 'X';
    ew_r <= '1'; ew_y <= 'X'; ew_g <= 'X';

end case;
end process;

```

```

-----
-- Next-state logic
-- ALL states now last 1 second.
-----

```

```

next_state_decode : process(state, timer)
begin
    next_state <= s0;
    timer_reset <= '0';

```

```

-----
-- State machine (Lab 9.3.1 basic flowchart)
-- These 6 states come from the FIRST diagram
-----

```

```

case state is

```

```

-- s0: NS green
when s0 =>
    if timer = 1 then
        next_state <= s1;
        timer_reset <= '1';
    else
        next_state <= s0;

```

```

    end if;

-- s1: NS yellow
when s1 =>
    if timer = 1 then
        next_state <= s2;
        timer_reset <= '1';
    else
        next_state <= s1;
    end if;

-- s2: all red
when s2 =>
    if timer = 1 then
        next_state <= s3;
        timer_reset <= '1';
    else
        next_state <= s2;
    end if;

-- s3: EW green
when s3 =>
    if timer = 1 then
        next_state <= s4;
        timer_reset <= '1';
    else
        next_state <= s3;
    end if;

-- s4: EW yellow
when s4 =>
    if timer = 1 then
        next_state <= s5;
        timer_reset <= '1';
    else
        next_state <= s4;
    end if;

-- s5: all red
when s5 =>
    if timer = 1 then
        next_state <= s6;
        timer_reset <= '1';
    else

```



```
    next_state <= s5;
end if;
```

```
-- ADDITIONAL STATES BELOW ARE FROM FLOWCHART #2 (Lab 9.3.2)
-- This block implements the extended timing and transitions
-- required by the SECOND diagram in the manual.
```

```
-- s6: (Flowchart #2) NS left-turn green
when s6 =>
    if timer = LT_GREEN_TIME then
        next_state <= s7;
        timer_reset <= '1';
    else
        next_state <= s6;
    end if;
```

```
-- s7: (Flowchart #2) NS left-turn yellow
when s7 =>
    if timer = LT_YELLOW_TIME then
        next_state <= s8;
        timer_reset <= '1';
    else
        next_state <= s7;
    end if;
```

```
-- s8: (Flowchart #2) all red transition before EW sequence
when s8 =>
    if timer = ALL_RED2_TIME then
        next_state <= s3;
        timer_reset <= '1';
    else
        next_state <= s8;
    end if;
```

```
-- Fallback
```

```
when others =>
    next_state <= s0;
```

```
end behav;
```

Schematic/Diagram/Circuit:

Figure 9.4: Flowchart for the Simple Traffic Light Controller

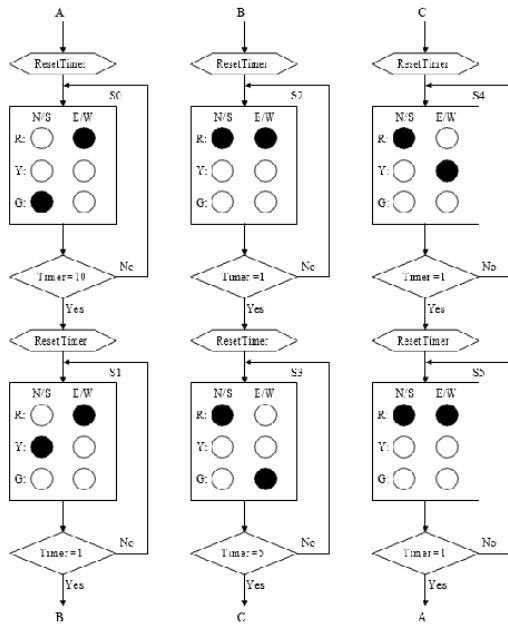
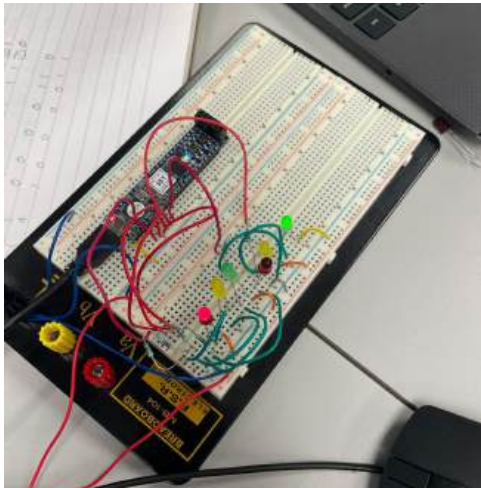
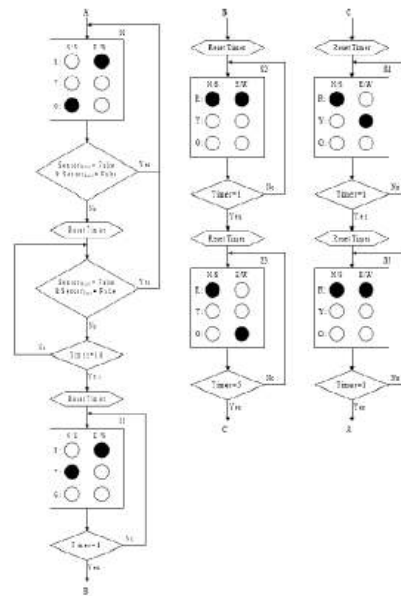


Figure 9.6: Howchart of the Traffic Light Controller with the Automobile Sensors Added



VII. References

[1] Getting Started with Vivado,

<https://docs.xilinx.com/v/u/2018.2-English/ug910-vivado-getting-started>

[2] Digilent Inc., 'CMDO A7 Reference Manual',

https://digilent.com/reference/_media/cmod_a7/cmod_a7_rm.pdf

[3] Mano, Morris M. Digital Design. New Jersey: Prentice Hall.

[4] Wakerly, John F. (2001). Digital Design Principles & Practices Third Edition. New Jersey: Prentice Hall.

[5] ECE 218 Lab Manual with FPGA Experiments